

LLM-Driven Approach to Automated Sustainability of IoT Systems

Nenad Petrović
Department of Computer Science
Faculty of Electronic Engineering
University of Niš
Niš, Serbia
nenad.petrovic@elfak.ni.ac.rs
ORCID: 0000-0003-2264-7369

Suad Suljović
Faculty of Information Technologies
Metropolitan University
Niš, Serbia
ssuljovic@atssb.edu.rs
ORCID: 0000-0003-1822-3782

Dragana Krstić
Faculty of Electronic Engineering
University of Niš
Niš, Serbia
dragana.krstic@elfak.ni.ac.rs
ORCID: 0000-0002-2579-3911

Dario Javor
Department of Systems Research in Safety
and Risk
Faculty of Occupational Safety
University of Niš
Niš, Serbia
dario.javor@znrfak.ni.ac.rs
ORCID: 0000-0002-6089-3555

Abstract—Due to huge number and heterogeneity of devices within Internet of Things (IoT)-based architectures, scalability and maintainability of such systems rise many challenges when it comes to long-term sustainability, especially regarding scalability and maintainability aspects. In this paper, we explore state-of-art Large Language Model (LLM)-based techniques as well as Agentic AI approaches in order to automate these aspects and reduce the cognitive load and need for human intervention. As intermediary step for system representation, we leverage Model-Driven Engineering (MDE) approach in order to ensure verifiability. As outcome, two case studies relying on GPT-4o model are presented.

Keywords—IoT systems, large language model, model-driven engineering

I. INTRODUCTION

Currently, IoT-based systems and solutions are often used as foundation for many use cases affecting our everyday life – from smart home solutions to traffic management and public infrastructure in smart cities. However, over time, considering the new requirements and innovative use cases coming from increasing customer demands, such systems need to evolve. Either by introducing new types of devices or topology modifications that might be required for certain scenarios, these systems should be extendable on long-term perspective. On the other hand, handling the system which consists of increasing number of heterogeneous devices in case of unexpected events becomes increasingly difficult as well. Therefore, scalability and maintenance of such systems represent highly relevant aspects enabling IoT system sustainability [1]. Regarding the scalability – each time new device has to be added, many factors have to be taken into account in order to provide its seamless integration. Apart from making sure that it receives/sends the expected messages and input/output data formats are compatible with other devices, it still needs to be considered if device is suitable for

achieving system's goals when it comes to the perceived Quality of Service (QoS) from perspective of end-users. On the other side, various events that occur across different phases of IoT system's lifecycle, especially failures, anomalies, topology changes and performance drops, potentially introduce bottlenecks when it comes to overall usage experience and functionality offered [1]. In such cases, usually a set of maintenance-related actions has to be taken in order to reduce their negative impact. However, considering the underlying complexity of the infrastructure, such operation could require high cognitive load and significant amount of time, which might further affect system's usability and even lead to catastrophic outcomes in case of sensitive domains and usage scenarios (such as traffic management and public safety-related).

Therefore, in this paper, we explore the potential of trending Large Language Models (LLMs) when it comes to automation of aspects relevant to sustainability of IoT systems [2]-[4]. As outcome, we propose two case studies: 1) hardware abstraction with scalability, and 2) automatic maintenance. For experiments that involve automated code generation, we rely on OpenAI's GPT-4o. Moreover, advanced planning and reasoning capabilities of OpenAI's o1 family of models [5] are taken into account for maintenance scenario as well. Additionally, model-driven engineering (MDE) approach is used for formal IoT system representation and verification, relying on Eclipse Modeling Framework (EMF) and Object Constraint Language (OCL) rules, as shown in [6].

II. BACKGROUND AND RELATED WORKS

In the existing literature, LLMs show potential of adoption for similar case studies. In [7], an approach to maintenance and unforeseen event mitigation in industrial environment relying on synergy of LLMs and Augmented Reality (AR) is proposed. Work presented in [8] leverages LLMs for predictive maintenance of industrial machines relying on Raspberry Pi-based IoT nodes. In a similar way, [9] makes use of LLMs for IoT-based smart building maintenance aiming energy optimization.

While LLMs exhibit strong summarization and text generation capabilities, we make use of MDE in order to

This paper is supported by the Ministry of Science, Technological Development and Innovation of the Republic of Serbia pursuant to agreement with the University of Niš, Faculty of Occupational Safety [Grant Number: 451-03-137/2025-03/200148; Goal 7] and agreement with the University of Niš, Faculty of Electronic Engineering [Grant Number: 451-03-137/2025-03/200102].

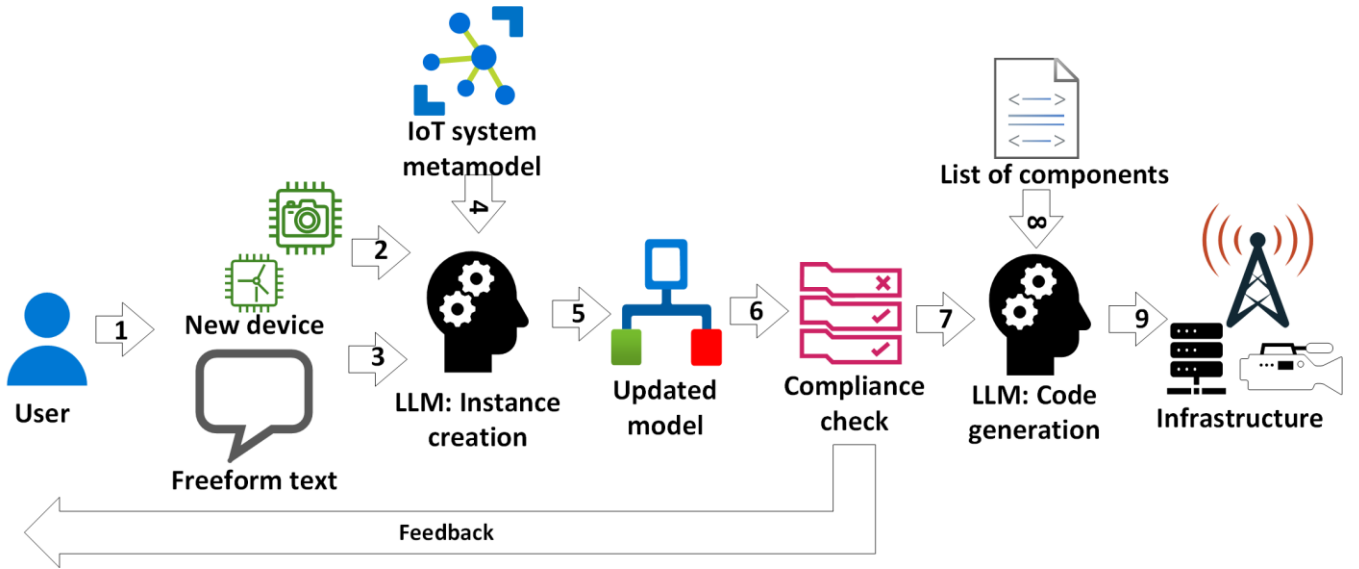


Fig. 1. Hardware abstraction and scalability case study: 1) user input; 2) device description; 3) device usage scenario; 4) Ecore metamodel; 5) XMI model instance; 6) pre-defined OCL-rules; 7) Check pass; 8) Docker commands, MQTT topics, drivers 9) target commands.

enable convenient system representation and verifiability of intermediate results before code generation, building upon our previous works [6, 10]. This is one of main advantages compared to existing solutions. In MDE, metamodels are used to describe crucial concepts, their attributes and relations within particular domain (IoT systems in our case). Furthermore, instances of model are created with respect to defined metamodel in order to represent the considered IoT system. In our previous works with elements of IoT device interoperability [1, 10], such steps assumed manual efforts, while in this paper we also aim to automate it. Additionally, design constraint rules (OCL in our case) are introduced in order to check whether certain conditions are satisfied within the system, inspired by [6]. Finally, we presented agentic AI-based network management approach relying on LLMs and model-based representations in [11], considering communication channel characteristics and environment impact on the perceived Quality of Service (QoS).

III. APPROACH AND IMPLEMENTATION OVERVIEW

A. Scalability and Hardware Abstraction

The proposed workflow for the first case study is depicted in Fig. 1. In the first step, we assume that user wants to add new IoT device to the existing system. The specification of new device is provided as well as freeform text describing the intended purpose of device usage by user. Afterwards, we make use of LLM to update the current system model based on text specification and IoT system metamodel (based on [1] and [8]). As outcome, updated system model is constructed. A set of design constraints is checked against the new system model. In case that all of the constraints are satisfied, we proceed with LLM-based code generation which takes into account new system's model-based representation and catalog of available pre-defined components. Such components cover various aspects to hardware abstraction and interoperability: 1) list of device-specific drivers for target host architecture 2) publish/subscribe message exchange topics relying on communication middleware 3) Docker containers list. Finally, the generated commands are executed against the target IoT infrastructure.

In this case, there are two underlying prompts common for all scenarios of this type:

Prompt 1: "Update system model [current] based on [user requirement]."

Prompt 2: "Create OCL rules with respect to [metamodel] for given [constraint]."

While the first one ensures that current system model is updated based on new requirements, the second one is leveraged to generate rules that determine whether system satisfies guidelines or reference architecture specification.

B. Automated Maintenance

In Fig. 2, the underlying workflow for the second case study is shown.

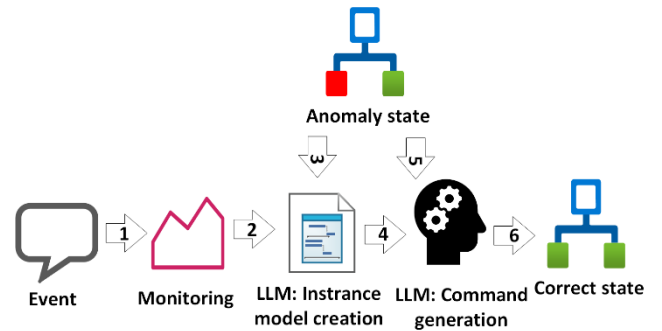


Fig. 2. Automated maintenance case study: 1) event log record; 2) anomaly; 3) and 5) XMI model instance; 4) decision list 5) Hardware abstraction and scalability case study: 1) user input 2) device description 3) device usage scenario 4) Ecore metamodel 5) XMI model instance 6) pre-defined OCL-rules 7) Check pass 8) Docker commands, MQTT topics, drivers 9) target commands.

It is assumed that relevant events, such as failures, anomalies, performance drops and topology changes are monitored. In case of such events, AI-based reasoning and planning agent based on ol-mini is triggered to decide on maintenance action based on current system representation (model instance). LLM-based planning agent decides sequence of steps to be taken as a response in order to keep the system healthy and its performance satisfiable, based on

set of pre-defined actions. Finally, the selected actions are parametrized in order to generate commands executed against the infrastructure.

IV. EXPERIMENTAL SCENARIOS AND RESULTS

For purpose of evaluation, we rely on physical environment which consists of Raspberry Pi Model 3B devices [12, 13] as IoT processing nodes. They are part of mixed architecture (x86/ARM) Kubernetes cluster (as described in [1]). Raspberry Pi device itself is equipped with 64bit quad-core ARM 1.2GHz Broadcom BCM2837 CPU and 1GB of RAM. Additionally, each Raspberry Pi was also running Docker Engine. While in the first two experiments we consider them as stand-alone devices, within the third experiment, we consider Raspberry Pi Model 3B as part of Turtlebot [14] mobile robot.

As underlying LLM, we make use of commercial, pre-trained version of OpenAI's GPT-4o [15] without fine-tuning applied in synergy with o1-mini [5] model for complementary capabilities of reasoning. In order to show the generality of the proposed approach, two different communication protocols were considered: ROS 2 (Robot Operating System 2) [16] and MQTT (Message Queuing Telemetry Transport) [17]. While both of them are publish-subscribe based, ROS2 is a middleware framework specialized for robotics applications, relying on DDS (Data Distribution Service) under the hood. On the other side MQTT is a lightweight messaging protocol designed for IoT and constrained devices, providing publish-subscribe messaging over TCP/IP with brokers, while prioritizing low bandwidth and reliability over high performance.

Table I shows illustrative scenarios, underlying prompts, generated outputs, response times (denoted as *Resp.*) and hallucination rate (denoted as *Hal.*, based on 5 executions). There are two hardware abstraction scenarios (denoted as A-1 and A-2) and one maintenance scenario (denoted as B-1).

Based on the achieved results, it can be noticed that execution time is order of magnitude of 10s in all the experiments. It should be also mentioned that the previously mentioned prompts 1 and 2 are used in both A-1 and A-2.

Regarding the occurrence of hallucinations, it is observable that some of the values which are not explicitly specified (like specific communication protocol and service port numbers, container, drivers and MQTT/ROS2 topic names) would likely cause hallucinations within 5 runs. However, in the second experiment, where all the values are already explicitly specified, no hallucinations were detected. On the other side, the third experiment does not specify explicitly which approach to use to achieve the desired goal (as there are many alternatives - various additional Linux libraries and driver-specific tools), which causes more frequent occurrence of hallucinations, especially when it comes to driver and tool names referenced within the generated commands.

Regarding the execution time, the second experiment (B-1) has the longest processing duration, considering the fact that we make use of complementary o1-mini based component for decisioning in order to select the appropriate action in given context, apart from GPT-4o prompt execution in order to generate the commands that should be executed.

On the other side, manual procedures of IoT-based system extension with new hardware, reconfiguration and

maintenance require more time [1], usually resulting with procedures whose duration is at least order of magnitude greater than one minute.

TABLE I. PROMPTS AND SCENARIO EXAMPLES

Case	Prompt/Scenario Example	Outcome	Resp. [s]	Hal.
A-1	User requirement: Add new D435 camera with specification: 25fps, 1920x1080 for vehicle detection. System prompt:	MQTT: client.connect("mqtt_broker_address", 1883, 60) client.subscribe("vehicle_detection") Docker: docker run -it --rm \\\n--privileged \\\n--device=/dev/bus/usb \\\nvehicle_detection_d435	~25s	1/5
	Prompt 3: Generate corresponding MQTT python commands for [user requirement] considering the list of all topics [MQTT topics list]			
	Prompt 4: Generate corresponding Docker command for [user requirement] considering the list of all Docker images available [Docker images]	OCL: context VehicleDetectionCamera inv MinimumRequirements: self.frameRate >= 20 and self.resolutionWidth >= 1280 and self.resolutionHeight >= 720		
B-1	Prompt 5: Your task is to react in case of IoT system anomalies and changes. Possible actions are install_driver, restart_device, subscribe_topic, run_service, scale_up. In case of scale_up, create 3 replicas. Event:vehicle_detection failure	Action: scale_up Command: kubectl scale deployment vehicle-detection --replicas=3	~30s	0/5
A-2	User requirement: Limit Turtlebot's LIDAR to 30 degrees. The collected data via LIDAR on Turtlebot should be published to ROS2 topic called	Python script - lidar_camera_fusion/launch/lidar_remap.launch.py: from launch import LaunchDescription from launch_ros.actions import Node	~20s	

Case	Prompt/Scenario Example	Outcome	Resp. [s]	Hal.
	<i>lidar_camera_fusion.</i>	def generate_launch_description(): return LaunchDescription([Node(package='turtlebot3_brain', executable='robot.launch.py', output='screen', remappings=[('/scan', '/lidar_camera_fusion')])])		2/5
	Prompt 6: Generate ROS2 commands for Turtlebot based on [user requirement] assuming the list of available topics [ROS2 topics list]			
	Constraint: LIDAR should have angle larger than 10 degrees.	ROS2 commands: ros2 param set /lidar_node angle_min -0.2618 # -15° ros2 param set /lidar_node angle_max 0.2618 # +15° ros2 launch lidar_camera_fusion lidar_remap.launch.py		

V. CONCLUSION

Based on the achieved results, LLMs exhibit strong potential to both hardware abstraction and maintenance of IoT systems. While for the scenarios in case of hardware abstraction, the achieved execution time is satisfactory for its purpose, for maintenance - faster response might be expected for time-critical scenarios in sensitive domains. However, for all of the presented experiments, the benefits are noticeable compared to manual procedures, especially due to fact that cognitive load is reduced, as well as the order of magnitude of time needed.

In future, consideration of locally deployable, fine-tuned LLMs will be considered, as in this paper we rely on pre-trained solutions. Additionally, we will also take into account hallucination-tackling techniques in order to reduce the rate of their occurrence and negative impact on the final outcome. Finally, integration with agentic AI approach and LLMs-based techniques considering IoT security aspects will be also taken into account [18].

REFERENCES

- [1] N. Petrović, M. Tošić, "SMADA-Fog: Semantic model driven approach to deployment and adaptivity in fog computing," SIMPAT vol. 101, p. 102033, 2020.
- [2] H. Al-Safi, H. Ibrahim, P. Steenson, "Vega: LLM-driven intelligent chatbot platform for internet of things control and development," Sensors, vol. 25, p. 3809, 2025.
- [3] P. Kumar, "Large language models (LLMs): Survey, technical frameworks, and future challenges," Artif. Intell. Rev., vol. 57, p. 260, 2024.
- [4] Y. Liao, E. de Freitas Rocha Loures, F. Deschamps, "Industrial internet of things: A systematic literature review and insights," IEEE Internet Things J., vol. 5, pp. 4515-4525, 2018.
- [5] K. Wang, J. Li, N. P. Bhatt, Y. Xi, Q. Liu, U. Topcu, and Z. Wang, "On the planning abilities of OpenAI's o1 models: Feasibility, optimality, and generalizability," arXiv preprint, arXiv:2409.19924, Sep. 2024.
- [6] D. Krstić, S. Suljović, G. Đorđević, N. Petrović, and D. Milić, "MDE and LLM synergy for network experimentation: case analysis of wireless system performance in Beaulieu-Xie fading and κ - μ cochannel interference environment with diversity combining," Sensors, vol. 24, no. 10, p. 3037, 2024.
- [7] J. Angelopoulos, C. Manettas, and K. Alexopoulos, "Industrial maintenance optimization based on the integration of Large Language Models (LLM) and Augmented Reality (AR)," Advances in Artificial Intelligence in Manufacturing II, Lecture Notes in Mechanical Engineering, 2025, pp. 197-205.
- [8] K. R. Prabha, B. Nataraj, S. Sathish, C. Sujith, and G. Suthakarr, "Predictive maintenance for industrial machinery using LLM," Int. Res. J. Adv. Eng. Hub (IRJAEH), vol. 03, no. 05, pp. 2063-2069, May 2025.
- [9] Y. Li et al., "A large language model-based building operation and maintenance information query," Energy and Buildings, vol. 334, no. 2, p. 115515, May 2025.
- [10] N. Petrović, S. Konićanin and S. Suljović, "ChatGPT in IoT systems: Arduino case studies," Proc. of the IEEE 33rd International Conference on Microelectronics (MIEL), Nis, Serbia, 2023, pp. 1-4.
- [11] S. Suljović, N. Petrović, V. Vujović, M. Milašinović, G. Đorđević, and R. Stefanović, "AI agent-driven maintenance: Case study of outage probability for 5G wireless system with L-branch SC receiver influenced by rician fading and rician co-channel interference," Proc. of the 12th International Conference on Electrical, Electronic and Computing Engineering (IcETRAN), Čačak, Serbia, 2025, pp. 1-5.
- [12] E. Kadiyala, S. Meda, R. Basani, and S. Muthulakshmi, "Global industrial process monitoring through IoT using Raspberry pi," Proc. of the International Conference on Nextgen Electronic Technologies: Silicon to Software (ICNETS2), Chennai, India, March 2017, pp. 260-262, 2017.
- [13] J. Smith, and M. Davis, "Testing and debugging IoT projects with Raspberry Pi," J. Internet Things, vol. 8, pp. 123-135, 2020.
- [14] D. Singh et al., "TurtleBot: Design and hardware component selection," Proc. of IEEE International Conference on Computing, Power and Communication Technologies (GUCON), Greater Noida, India, 2018, pp. 805-809.
- [15] R. Islam, and O. M. Moushi, "GPT-4o: The cutting-edge advancement in multimodal LLM," in Intelligent Computing, CompCom 2025. Lecture Notes in Networks and Systems, Springer, vol. 1426, 2025.
- [16] R. Carreira et al., "A ROS2-based gateway for modular hardware usage in heterogeneous environments," Sensors, vol. 24, no. 19, p. 6341, 2024.
- [17] B. Mishra, and K. Attila, "The use of MQTT in M2M and IoT systems: A survey," IEEE Access, vol. 8, pp. 201071-201086, 2020.
- [18] N. Petrović, D. Krstić, S. Suljović, S. Hanczewski, and M. Glabowski, "Agent-Based AI approach to security in IoT systems leveraging GenAI," Proc. of the 33rd Int. Conf. Software, Telecommunications and Computer Networks (SOFTCOM), Split, Croatia, 2025.